内核设计优化实现文档

BuildStorm 场景下 MossOS 全链路优化

参赛院校： 重庆邮电大学
参赛队名： 每天睡满八小时
队伍成员： 吴勋、张可欣
指导教师： 马彬、钟福金

2026 年 **8** 月

目录

1 概述 1
2 问题定位与根因分析 2
2.1 负载特征 2
2.2 火焰图定位问题 2
2.3 与 Linux 基线的同口径对比 3
2.4 根因分析 4
3 优化设计与实现 8
3.1 优化一：per-CPU 物理页缓存（PCP） 8
3.2 优化二：地址空间“准备/执行/提交”三段式 8
3.3 优化三：Folio 系列（folio 页缓存与只读文件页直接映射） 9
3.4 优化四：ext4 锁外数据读写 + VirtIO 多在途请求 10
3.5 优化五：mprotect 按 VMA 定位并批量修改权限 11
3.6 优化六：TLB 刷新系列（异步回收、合并 TLB 刷新与定向刷新） 11
3.7 其他优化 12
4 实验分析 13
4.1 实验环境与方法 13
4.2 端到端结果 13
4.3 逐项优化分析 14
4.4 CPU 并行度变化 15
4.5 内存与 I/O 16
4.6 历史第一轮整包数据 16
4.7 结果可信度与限制 16
5 AI 使用说明与可复现步骤 17
5.1 AI 使用说明 17
5.2 可复现步骤 17
6 总结与展望 20

第 1 章 概述

BuildStorm 题目要求内核在 QEMU 模拟的 RISC-V64 与 LoongArch64 平台（`-smp 8 -m 8G`）上离线从源码全量编译 `arceos-helloworld`，这是一次数百 crate 的真实 Rust 编译，伴随重度文件读写、多线程与大量 `mmap/mprotect` 系统调用。评分上，编译成功为门槛，加速分按编译耗时与同一评测机 Linux 基线 `B` 的关系计算，因此能否完整跑通并显著快于两倍 Linux 基线直接决定本题得分。

针对这一目标，我们首先在带 guest profiler 的内核上完整运行 BuildStorm，通过火焰图与路径探针定位问题与根因。结果显示瓶颈集中在文件缺页、页缓存与文件系统 I/O 链路，主要问题包括：

- 缺页处理持有地址空间独占锁等待磁盘 I/O；
- 物理页分配与释放集中在全局分配器；
- 只读文件映射多做一次私有页分配与复制；
- 页缓存按单页组织，逐页查找、重复读盘；
- ext4 全局锁内等待块设备；
- `mprotect` 反复扫描全局映射记录。

这些全局锁、逐页处理和重复工作使 8 个 vCPU 的并行能力无法发挥（优化前 6～8 核时间 占比仅约 6%）。围绕上述根因，我们完成了以下关键优化：

- per-CPU 物理页缓存（PCP），缓解全局分配器竞争；
- AddrSpace 三段式缺页（prepare / execute / commit），慢操作移出地址空间锁；
- folio 页缓存，合并相邻页的加载与 I/O；
- 只读文件页直接映射，真正写入时才 COW；
- ext4 数据 I/O 移出全局锁，配合异步 VirtIO 多在途请求；
- per-VMA `mprotect`，只更新目标区间；
- 后续补充：VFS/inode 元数据并发、异步回收与跨 VMA TLB 合并、精确 TLB shootdown。

实验数据均为本地测试所得（采用 QEMU `virt`、8 vCPU、8 GiB、同一磁盘镜像、关闭 guest profiler），受本地电脑配置限制，实验数据会与测评机实际配置存在偏差，本地数据主要 用于横向比较各优化阶段的效果。在此口径下，优化前 BuildStorm 正式编译耗时为 1524.50 秒；六项核心优化完成后降至 628.53 秒（-58.77%，约 2.43 倍加速）；继续完成 后续优化后进一步降至 460.76 秒（累计 −69.78%）。平均有效核数由 3.252 核提高到 4.410 核，6～8 核时间占比由 6.23% 提高到 42.93%。

目前，Moss 内核已在评测机上完成 BuildStorm 测例的正式测评，RISC-V64 与 LoongArch64 两侧得分均为 **180** 分。排行榜截图如下：

#	用户名	队伍	学校	提交次数 (ASC)	buildstorm-glibc-la	buildstorm-glibc-rv	cagent-glibc-la	cagent-glibc-rv	rank
1	T202610617999569	每天睡满八小时	重庆邮电大学	44	180.0	180.0	199.1	199.1	758.2000
2	T202612332999758	这行代码有问题	烟台南山学院	76	180.0	180.0	199.1	199.1	758.2000
3	T2026104869910383	洗剪吹	武汉大学	73	180.0	179.0	199.1	199.1	757.2000
4	T2026181239910993	Unicus	哈尔滨工业大学（深圳）	17	180.0	150.6	199.1	199.1	728.8000

图 1　BuildStorm 测评成绩截图（截至 2026.8.5）

第 2 章 问题定位与根因分析

2.1 负载特征

BuildStorm 的正式编译任务由构建助手 `tg-xtask` 驱动，先进行 446 个 crate 的预构建，再执行正式目标构建。从内核视角看，这一负载有以下特征：

- 小文件密集：数百个 crate 源码、`.rlib`、增量产物和临时文件，大量文件以 `MAP_PRIVATE` 方式映射并顺序读取，单文件多为 4 KiB 页的整数倍；
- 多线程高并发：cargo/rustc 并行调度，8 个 vCPU 上同时存在多个编译任务与页缺失；
- 进程生命周期短：rustc、链接器、构建脚本频繁 fork/exec/exit，地址空间反复建立与销毁，`exec` 需要整体切换地址空间；
- 内存映射系统调用频繁：`mmap/munmap/mprotect` 被大量调用，其中 `mprotect` 尤其密集；
- 文件系统 **I/O** 重：冷缓存时大量页缓存 miss 需要进入 ext4 查询块映射并等待块设备。

这些特征叠加在一起，使得“单次系统调用本身的耗时”并不重要，重要的是缺页、页缓存、文件系统与内存管理路径上的全局锁竞争和串行等待被放大。

2.2 火焰图定位问题

我们在带 guest profiler 的内核上完整运行 BuildStorm，对每个 vCPU 以 10 Hz 采样内核/用户栈，并对 mutex 等待、ext4 全局锁等待、页缓存与同步块 I/O 等事件挂接探针，生成 CPU 火焰图与等待类火焰图，分别反映时间的消耗热点与线程的阻塞热点。

需要说明的是，这一火焰图方法并非仅在优化前运行一次：每完成一轮关键优化后，我们均使用相同的 profiler 配置重新采集火焰图，通过前后对比来判断优化是否命中了目标热点、瓶颈是否发生了迁移。报告中引用的数值如无特别说明，均来自优化前基线（`riscv-buildstorm-guest-20260721`）；优化后的对比数据来自 2026-07-24 整包优化后的复测（`riscv-buildstorm-guest-20260724`），两份报告的实验配置与采样口径完全一致，具备同口径可比性。

赛题只计时 `cargo xtask arceos build -p arceos-helloworld`（即实验中的 timed 阶段，`elapsed_s`），因此分析聚焦 timed 阶段。下图是优化前基线的 timed 阶段的 CPU 火焰图：

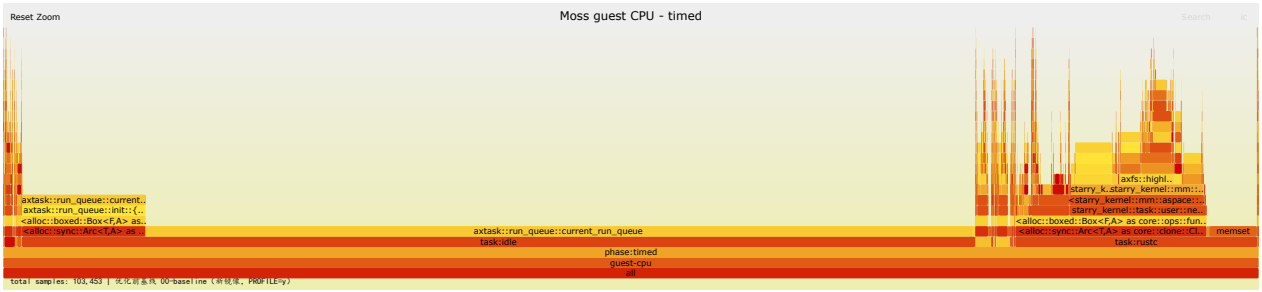


图 2　优化前 timed 阶段 CPU 火焰图

从 CPU 火焰图可以发现以下主要问题：

- 并行度低：`task:idle` 占全部采样的 75.9%，非 idle 仅 24.1%；
- 内核时间集中在缺页路径：最宽的塔是 `BackendOps::populate ->CowBackend::alloc_new_at -> alloc_new_frame -> alloc_frame`，其上方还挂着 `with_page_or_insert_impl`、`page_or_insert`、`PageCache::new` 等页缓存函数；
- 锁/关中断/清零开销明显：叶函数热点见下表。

热点	样本数	占比
NoPreemptIrqSave::release	10252	41.19%
lwext4 memset	4748	19.07%
virtio VirtQueue::can_pop	1137	4.57%
core::ptr::read	749	3.01%
BTreeMap navigate	500	2.01%
disable_preempt	450	1.81%
MemorySet::protect	230	0.92%

表 2-1　优化前 timed 阶段主要 CPU 热点（叶函数，占非 idle 样本）

mutex 等待火焰图如下，最宽的塔是 `task:rustc -> new_user_task -> BackendOps::populate -> CowBackend::alloc_new_at`：rustc、lto、opt 等编译任务经这条 COW/文件缺页路径的等待合计约 89%。也就是说，缺页线程在持有进程 `AddrSpace` mutex 时进入文件读取，再被页缓存与 ext4 的 mutex 挡住，层层排队。

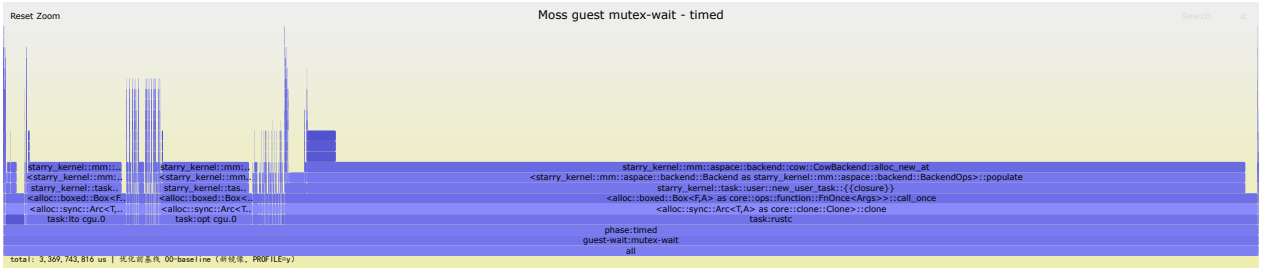


图 3　优化前 timed 阶段 mutex 等待火焰图

页缓存与文件系统的时间去向由事件探针给出：

事件	累计值
page-cache inclusive	4042.25 s
mutex wait	3369.74 s
ext4 inclusive	453.85 s
精确 ext4 全局锁等待	103.89 s
同步块读取	170.38 s
同步块写入	6.23 s
all off-CPU	70132.78 s

表 2-2　优化前 timed 阶段路径累计值（跨任务累计，可大于墙钟）

其中 `page-cache inclusive` 占据主导（单个 rustc 缺页栈占 70.82%，各任务合计约 84%），最宽路径仍是缺页；ext4 全局锁等待 103.89 s 中最大的单一路径同样是缺页读取（79.16%）。CPU、mutex 等待与页缓存/ext4 三组数据交叉指向同一条热点链：”缺页 → `AddrSpace` 大锁 → 页缓存 → ext4 → 块设备”，具体性能热点在下一节逐项展开。

2.3　与 Linux 基线的同口径对比

为了更清晰直观的看原来的 Moss OS 的性能缺陷，我们以相同事件模型对 Linux guest 重新采样，得到了当时的对照：

指标	**Linux**	**Moss**（优化前）	差距
正式阶段	454.22 s	2524.89 s	约 5.56 倍
锁等待（预构建/正式）	1.503 / 1.251 s	11166.94 / 8172.23 s	约 6500～7400 倍
page-cache inclusive（预构建/正式）	53.57 / 22.00 s	15042.35 / 9343.23 s	约 280～425 倍
前台同步 I/O（预构建/正式）	6.386 / 2.673 s	456.66 / 187.39 s	约 70 倍
ext4 read/write（预构建/正式）	70.05 / 22.49 s	2222.89 / 682.54 s	约 30 倍

表 2-3　Moss（优化前）与 Linux 的 BuildStorm 同口径对比

Linux 的活跃核等价值为 6.20 / 5.48 核，而优化前 Moss 只有 3.68 / 2.36 核。通过对比可以说明 Moss 与 Linux 的差距主要集中在文件缺页链路（锁等待、page-cache、同步 I/O、ext4）而非编译器本身的执行效率，首要修复方向是拆除“AddrSpace → CowBackend → page-cache → ext4 → block”的跨层大临界区。

2.4　根因分析

在逐条分析热点之前，下图展示了优化前发生缺页的完整处理链路。图中左右两侧四根彩色竖线分别标注四种锁的持锁周期——AddrSpace 独占锁、cache_op + page_cache 锁、ext4 全局自旋锁、全局页分配器锁。

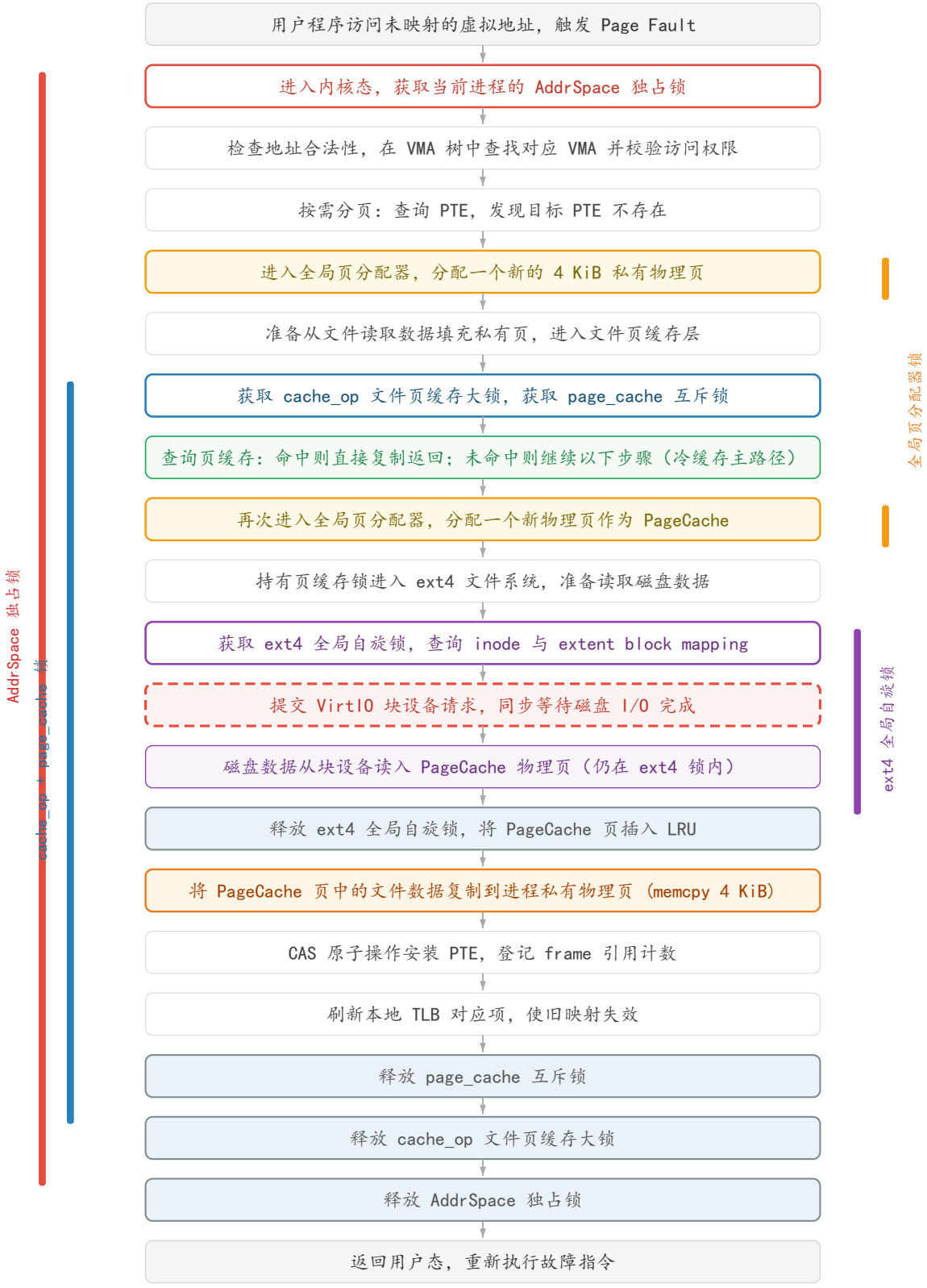


图 4　优化前文件缺页处理链路与锁范围

从图中就可以清晰地看到旧链路的两类结构性问题。

一方面是“大锁套小锁”：外层的 AddrSpace 锁尚未释放就逐层获取 page_cache 锁和 ext4 锁，内层锁上的任何阻塞都会直接穿透到外层的所有等待者。当一个线程在 ext4 锁内等磁盘 I/O 的时候，

不仅持有 ext4 锁不放，还同时卡住了 page_cache 锁和 AddrSpace 锁，其他 7 个核上所有需要这些锁的线程就全部都要排队。

另一方面是“锁粒度太大”：同一把锁将快操作和慢操作捆在一起——ext4 锁内既做微秒级的 extent 映射查询，也做毫秒级的同步磁盘等待；AddrSpace 锁内既有轻量的 VMA 查找，也有重量级的物理页分配和文件数据复制。一些不需要锁内进行的重任务也被纳入了锁内。

并且两类问题会互相放大：内层锁做着毫秒级的慢活，外层锁只能被动等待，而 BuildStorm 会同时运行成百上千个 rustc/cc1 进程、每个都在反复缺页读文件，8 个 vCPU 的并行能力被层层锁链压缩为实测仅 2.36 核的有效并行度，导致多核被串行化，难以发挥多核的效率。优化前 mutex 累计等待高达 8172.23 s（跨任务累计），其中约 74% 经过图中所绘的这条缺页锁链。

因此，后面将介绍的六项核心优化的本质方向是一致的：缩短持锁时间（三段式、ext4 锁外 I/O）、降低进锁频率（PCP、folio 合并）、缩小锁的管辖范围（per-VMA mprotect），每一项都在拆除这条锁链中的一环。

结合火焰图、探针累计值与代码阅读，将主要瓶颈归纳为以下六个根因：

2.4.1 根因一：缺页处理持有整个地址空间的独占锁等待慢操作

在旧链路中，一次文件缺页会独占当前进程的 `AddrSpace`，在锁内完成 VMA 查找、物理页分配、页缓存 miss、ext4 块映射查询、磁盘 I/O 等待、数据复制与 PTE 安装。磁盘 I/O 的等待时间远大于 VMA 查询本身；一个线程等盘时，其他线程即使缺的是完全不同的文件或地址，也只能排队。BuildStorm 线程多、文件多，把这种串行等待放大是最主要的性能瓶颈。优化前火焰图中，约 74% 的 mutex 累计等待（8172.23 s，timed 阶段）会经过 `BackendOps::populate → CowBackend::alloc_new_at` 这条 COW 文件缺页路径。

2.4.2 根因二：物理页分配与释放集中在全局分配器

缺页分配私有页、页缓存分配页、COW 分配页以及进程退出时释放页，都会进入全局物理页分配器并竞争同一把锁。8 个 vCPU 同时缺页时，所有 CPU 的分配和释放都被串行化。优化前 CPU 火焰图中 `NoPreemptIrqSave::release` 约占非 idle 样本的四成以上，是排名第一的叶函数热点。

2.4.3 根因三：MAP_PRIVATE 只读缺页多一次私有页分配与复制

对于只读使用的 `MAP_PRIVATE` 文件映射，旧实现即使只是读取，也会”先分配页缓存页、读盘、再为进程分配一张私有物理页、从页缓存复制 4 KiB、安装私有页 PTE”。大量只读文件页永远不会被写，私有页分配和 4 KiB 复制完全浪费，且每个进程映射同一文件都会各复制一份，加剧内存压力与缺页成本。这一问题主要来自代码逻辑分析——火焰图中 `CowBackend::alloc_new_at` 是最宽的缺页塔，而其中的 memcpy 和私有页分配正是该塔的组成部分，但火焰图本身无法直接区分”必要的复制”与”浪费的复制”。

2.4.4 根因四：页缓存粒度小，逐页查找、重复 I/O 与重复等待

旧实现页缓存按单页组织：一次 miss 只加载一张 4 KiB 页，相邻页需要再次进入页缓存查询并可能再次读盘；多个线程同时缺同一页时还可能重复发起 I/O。优化前探针显示 page-cache inclusive 累计 9343.23 s（timed 阶段），是量级最大的嵌套路径，火焰图中占据约 91.5% 的图宽，其中 71.85% 经过 `with_page_or_insert_impl`。

2.4.5 根因五：ext4 全局锁内等待块设备

Moss 用一把全局锁保护 `LwExt4Filesystem` 元数据。旧实现中一次文件读取在锁内完成块映射查询后继续等待磁盘 I/O 完成才释放锁，导致不同文件的数据读取被全局串行化。优化前探针显示 ext4 全局锁等待累计 218.92 s（timed 阶段），同步块读取 175.67 s；ext4 锁等待火焰图中最大的单一调用栈同样是 rustc 文件映射缺页（占 64.80%）。

2.4.6 根因六：mprotect 扫描全局映射记录与无关 VMA

旧实现维护所有进程共享的 `MMAP_RECORDS` 全局数组。一次 `mprotect` 需要扫描全局记录检查 seal/最大权限、修改页表后再次锁全局记录并重建数组；`MemorySet::protect` 则从地址空间头部遍历所有 VMA，而调用实际只覆盖一小段地址。旧插桩显示平均一次调用扫描约 3.16 万条全局记录，预构建阶段累计扫描约 897 亿条。

需要指出的是，在优化前火焰图中 mprotect 并非可见的主热点——此时缺页锁链造成的阻塞量级远大于全局记录扫描（`mprotect` 相关 mutex 等待仅占 1.18%），将其完全掩盖。这一问题由代码插桩而非火焰图发现。下图是缺页链路优化后重新采集的 mutex 等待火焰图，可以看到 mprotect 已成为压倒性的第一热点——占 mutex 累计等待的 95.16%、非 idle CPU 样本的 56.82%，是典型的“被更大瓶颈掩盖的独立串行化源”。

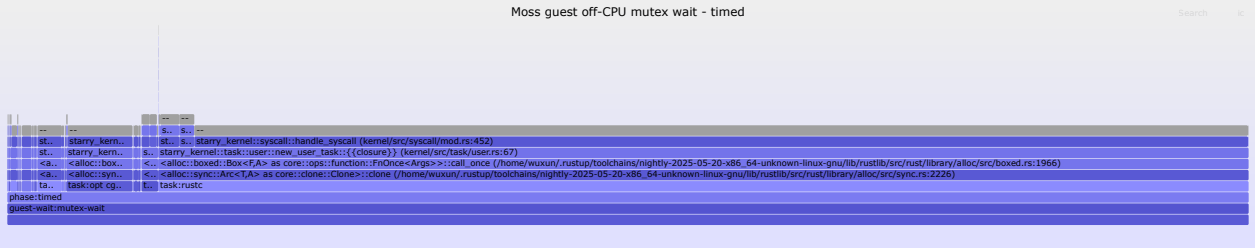


图 5　缺页链路优化后 timed 阶段 mutex 等待火焰图

第 3 章 优化设计与实现

本章按“六项核心优化 + 其他优化”两部分组织，六项核心优化按依赖顺序实施，其余针对 次生热点的改进归入“其他优化”。每项优化均遵循同样的工程约束：修改独立提交、保留 原始测量数据、先通过正确性测试再评估性能，避免把多项修改打包后无法归因。实现提交 与实验数据的对应关系见第 4 章。

3.1 优化一：per-CPU 物理页缓存（PCP）

如 2.4 根因所述，所有 order-0（4 KiB）物理页的分配与释放都集中在全局分配器， 8 个 vCPU 并发缺页时全局锁成为热点（火焰图中 `NoPreemptIrqSave::release` 占非 idle 采样的 41.19%）。为此在每个 CPU 身边维护一小撮空闲 4 KiB 物理页：

```
分配 4 KiB 页：
  先从当前 CPU 的 PCP pop
  -> 有：直接返回
  -> 无：从全局分配器批量取一组（refill），留下大部分，返回一页

释放 4 KiB 页：
  先 push 回当前 CPU 的 PCP
  -> 超过高水位：批量退还全局分配器（drain）
```

实现要点：

- `vendor/axalloc/src/pcp.rs` 提供 `PerCpuPageCache`，共 64 个 PCP slot，以 `cpu_id % PCP_CPU_SLOTS` 定位，只缓存 order-0 页；
- 空闲页的第一个机器字临时保存下一张空闲页地址（Moss 没有 Linux 的 `struct page` 链表节点），只有确认空闲且归分配器所有的页才能这样使用；
- PCP 按 `UsageKind` 分链，避免不同用途的统计与回收语义混在一起；
- `batch`/高水位控制一次从全局取多少或退多少；全局分配失败时 `alloc_page_cached` 会先 `drain_page_caches()` 把各 CPU 缓存退回全局再重试一次， 防止“假 OOM”并给高阶连续页合并创造机会。

正确性关键：多 CPU 并发分配/释放、高水位批量退页、内存压力下全量 drain、大块连续 内存申请以及不同 `UsageKind` 不串页；释放页的地址与对齐检查，正在使用的页绝不挂进 空闲链表。主要代码：`vendor/axalloc/src/lib.rs`（`alloc_page_cached`、`dealloc_page_cached`、`drain_page_caches`）、`vendor/axalloc/src/pcp.rs`。

3.2 优化二：地址空间“准备/执行/提交”三段式

如 2.4 根因所述，缺页在持有整个进程 `AddrSpace` 独占锁期间执行分配、页缓存、ext4 与磁盘 I/O，一个线程等盘时其他缺页、`mmap`、`munmap`、`mprotect` 全部排队（mutex 等待累计 3369.74 s，约 89% 经缺页路径）。因此把缺页处理拆成三段，慢操作全部移到 `AddrSpace` 锁外：

```
缺页 -> AddrSpace 读锁 prepare（查 VMA、校验权限、记录提交前提）
     -> 释放 AddrSpace
     -> execute（锁外分配页、查页缓存/folio、ext4 映射、等磁盘、COW 复制）
     -> AddrSpace 读锁 commit（重新查 VMA、重新校验前提、CAS 安装 PTE）
     -> stale 时有限重试，极端情况退回写锁慢路径
```

- **prepare**：持有读锁，只做“判断和开单”——定位 VMA、校验权限、检查 PTE 是否已被 其他线程装好、构造 `FaultPreparation`/fault plan，不读盘、不分配；

- **execute**：不持有 AddrSpace，并发执行物理页分配、页缓存查找/创建、等待 folio loader、进入 ext4、等待磁盘、COW 数据复制；
- **commit**：重新取得读锁，再次定位 VMA 并校验权限，确认 prepare 时的前提仍成立后 用原子 CAS 安装 PTE，同时登记 PTE 对应的 frame/folio 所有权。

提交结果分为 Handled（本线程成功或目标已被正确处理）、Stale（前提过期，需要 重试）与错误三类。实现用 MAX_CONCURRENT_FAULT_RETRIES = 4 限制乐观重试次数， 连续 stale 后回退到拿写锁的旧慢路径，保证极端竞争下仍能推进。

正确性关键：prepare/commit 使用读锁表示“VMA 树结构不被写者破坏、PTE 并发安装由 更细粒度的 CAS 协调”，结构性修改仍走写侧；fault plan 持有 execute 结果的所有权，CAS 失败或 VMA 改变时 Drop 必须释放白做的 frame/folio 引用；commit 不能直接使用 prepare 阶段的结果，因为 execute 期间映射可能已被 munmap/mprotect 或并发缺页 改变。主要代码：kernel/src/mm/access.rs、kernel/src/mm/aspace/mod.rs、 kernel/src/mm/aspace/backend/*.rs。

3.3 优化三：Folio 系列（folio 页缓存与只读文件页直接映射）

Folio 系列同时解决两个根因：页缓存粒度小导致的逐页查找与重复 I/O（根因四），以及 只读 MAP_PRIVATE 缺页多一次私有页分配与复制（根因三）。

3.3.1 folio 页缓存：分片、预读与回收

将文件页缓存的管理单位从单页提升为 folio（一组连续的 4 KiB 基础页，最大 order 为 4，即一个 folio 最多覆盖 16 页共 64 KiB）。用户页表仍按 4 KiB 安装 PTE，folio 只 让页缓存以更大单位进行查找、建缓存、读盘与预读、loading/uptodate 状态管理、回收与 引用固定：

- 页缓存索引分成 64 个 shard，降低所有缺页线程争抢同一把索引锁的概率；
- 同一个 folio 只允许一个 loader 真正读盘，其他线程找到同一 loading folio 后等待 其完成，避免重复读盘与重复填充；
- 每个文件的 FileAddressSpace 维护 folio 索引、每页锁、loading/uptodate/ truncated 状态、dirty/writeback 状态与 invalidate_lock（睡眠型读写锁）；
- 等待 loading folio 时必须先释放 invalidate_lock，完成后再重新加锁并检查同一页号 对应的是否仍是同一个 folio（等待期间 truncate 可能已替换 folio），避免“前台线程 持旧读锁等 loader、loader 被排队的写者挡住、写者等前台线程”的三方死锁；
- 用 FolioPageRef 保证已经安装进 PTE 的物理页在使用期间不会被提前回收。

一次 I/O 读入相邻多页、多个缺页共享一个 loader、元数据操作按 folio 摊薄，并为只读 直接映射与 fault-around 提供稳定引用。folio 并非越大越好：读入但从未访问的兄弟页会 浪费 I/O 与内存，因此保留单页 fallback，并根据 BuildStorm 顺序读特征选择 order。主要代码：vendor/axfs/src/highlevel/file.rs（FileAddressSpace、FolioPageRef、 page_ref_at）。

3.3.2 只读文件页直接映射（写时 COW）

对满足以下条件的缺页，直接把 folio 中的物理页以只读方式装入 PTE，不再分配私有页：

- 4 KiB 页；文件偏移与虚拟地址对齐；是完整文件页（非 EOF 尾部残页）；
- 本次不是写缺页；页缓存能返回稳定的 FolioPageRef。

```
文件 -> folio 页缓存页
                  |
                  +-> 进程 A 的只读 PTE
                  +-> 进程 B 的只读 PTE
                  +-> 进程 C 的只读 PTE
```

这仍然符合 MAP_PRIVATE 语义：PTE 必须只读，进程第一次写时再次缺页，分配私有页、从 folio 复制内容后以可写方式装入本进程，即 copy-on-write。

正确性关键在物理页生命周期：PTE 只保存物理地址，不会自动阻止页缓存回收该页，因此 安装只读 PTE 的同时必须保存 `FolioPageRef`——PTE 存在期间 folio 不可释放；PTE 移除 后要等远端 CPU 旧 TLB 失效才能放掉最后引用；truncate/invalidate 与 PTE commit 互相 校验；CAS 输掉的线程放掉自己的引用。缺少引用固定会产生最危险的错误：PTE 仍指向原 物理地址，但页已被回收并装入别的数据，用户进程静默读到错误内容。以下情况回退到“分配私有页并读取/复制”的安全慢路径：写缺页、非对齐读、EOF 不完整尾页、页缓存不能 返回稳定引用、大页或不支持的页大小、commit 时 folio 已失效。主要代码：`kernel/src/mm/aspace/backend/cow.rs`、`vendor/axfs/src/highlevel/file.rs`。

3.4 优化四：ext4 锁外数据读写 + VirtIO 多在途请求

如 2.4 根因所述，一次普通文件读取在 ext4 全局锁内完成块映射查询并等待磁盘，跨文件 I/O 被全局串行化（ext4 全局锁等待累计 103.89 s）。本优化分两层解除串行：extent cache 与锁外数据 I/O（上层），异步 VirtIO 多在途请求（下层）。

3.4.1 extent cache 与锁外数据 I/O

先区分两类工作：extent 映射查询属于短元数据操作，数据 I/O 属于长等待操作。优化后 读路径为：

```
read_mapped
  -> 检查请求是否按 ext4 block 对齐
  -> extent cache 命中：直接取得映射
  -> 未命中：短暂拿 ext4 锁
       -> map_blocks 查询逻辑块到物理块
       -> prepare_direct_data_access
       -> 释放 ext4 锁并保存 extent cache
  -> 根据物理块调用 disk.read_blocks（锁外等待磁盘）
```

extent cache 按 inode 分 64 个 shard，miss 时做约 1 MiB 的 lookahead；进入 ext4 锁 后还要再次检查 cache（排队期间其他线程可能已填好同一映射）。洞块没有物理地址，直接 填零。写路径更复杂：锁内分配/确认块映射并准备缓存一致性，锁外写数据块，最后再短暂 进入 ext4 提交文件长度与元数据；文件中间非对齐部分块写仍走 lwext4 的 read-modify- write 安全路径。任何可能改变块分配的操作（truncate、删除/重建 inode、分配新块的写、 会改变映射的元数据操作）必须失效对应 inode 的 extent cache。

3.4.2 异步 VirtIO 多在途请求

为避免“释放了 ext4 锁却在块设备层再用一把互斥锁串行等待整个 I/O”，同步加入异步 块设备层：

- 块设备接口提供非阻塞 `submit_read`/`submit_write`/`poll_complete`；
- VirtIO 驱动最多允许 16 个请求同时在途，设备状态锁只保护 descriptor 提交与回收， 不覆盖设备完成时间；
- 提交线程短暂持锁提交 descriptor 后释放锁，在自己的 `Completion` 上睡眠；独立 completion task 通过 IRQ/轮询回收 descriptor 并按请求 ID 唤醒对应线程；
- 队列满时在 `queue_space` 上等待，一个请求完成通常只唤醒一个提交者，避免惊群；
- `flush` 先等待全部在途请求排空再独占执行；系统启动早期 run queue/中断未就绪时仍 使用同步 I/O，进入 init 前才启用异步路径。

主要代码：`vendor/axfs/src/fs/ext4/fs.rs`、`vendor/axfs/src/fs/ext4/block_io.rs`、`vendor/axdriver_block/src/lib.rs`、`vendor/axdriver_virtio/src/blk.rs`、 `vendor/virtio-drivers/src/device/blk.rs`。

3.5 优化五：mprotect 按 VMA 定位并批量修改权限

如 2.4 根因所述，`mprotect` 需要扫描所有进程共享的全局 `MMAP_RECORDS` 与地址空间 中无关 VMA（平均一次调用扫描约 3.16 万条全局记录）。为此删除全局映射旁表，把权限 信息放进每个 VMA：

- 每个 `MemoryArea` 携带 `VmaMetadata`：`user_flags`（用户当前看到的权限）、 `may_flags`（允许提升到的最大权限）、`shared`、`file_backed`、`anonymous`、 `locked`；
- 当前进程的 `MemorySet` 用 `BTreeMap<VMA 起始地址, MemoryArea>` 保存 VMA，地址区间 需要有序查询与找相邻 VMA，BTreeMap 比哈希表更合适；
- 用 cursor 定位：在 `range(..=cursor).next_back()` 找覆盖 cursor 的最后一个 VMA， 检查其结束地址，然后 `cursor = min(VMA.end, end)`，直到覆盖整个 `[addr, addr+len)` 区间。复杂度近似 `O(log N + K)`，其中 `N` 为进程 VMA 数，`K` 为本次实际覆盖的 VMA 数。

`mprotect` 必须保证“要么全部成功，要么全部失败”，因此采用两遍式：

- 第一遍只校验：整段地址是否连续映射、每个 VMA 的 `may_flags` 是否允许目标权限、 哪些 VMA 实际需要变化；若目标权限已一致，直接走 no-op 快路径；
- 第二遍才修改：只更新相交的 PTE 子范围，必要时按左右边界分裂 VMA，更新目标段 flags/元数据，再把修改后完全兼容的相邻 VMA 合并，避免反复 `mprotect` 把地址空间 切碎。

正确性关键在 TLB 同步：修改 PTE 权限后其他 CPU 可能仍缓存旧 TLB（可写改只读时不 失效会继续允许写；不可执行改可执行涉及指令同步），因此页表修改、TLB shootdown 与 旧 frame/folio 引用释放必须遵守顺序，全程持有当前进程 `AddrSpace` 写侧保证 VMA 树 与页表不被结构性修改打断。主要代码：`vendor/memory_set/src/set.rs`（`MemorySet::protect`）、`kernel/src/syscall/mm/mmap.rs`。

3.6 优化六：TLB 刷新系列（异步回收、合并 TLB 刷新与定向刷新）

前五项核心优化把文件缺页与内存管理主链路缩短后，进程退出/exec 的地址空间清理与 TLB 同步成为新的可见热点。本系列从三个层面缩短这条路径：只处理实际驻留的页、合并 跨 VMA 的 TLB 刷新、只向真正使用目标页表的 CPU 刷新。

3.6.1 地址空间异步回收与仅扫驻留页

退出/exec 的前台线程把已死亡的 `AddrSpace` 交给回收队列，由多个回收 worker 独立 清理；清理按页表中实际存在的 PTE 前进（`next_present_mapping`），跳过空的高层节点 覆盖的整个地址范围，不再逐 4 KiB 探测虚拟空洞。

3.6.2 跨 VMA UnmapGather 合并 TLB 刷新

`UnmapGather` 把同一次 clear 中多个 VMA 的 frame/folio 引用暂存，最多累计 8192 个 驻留映射后统一执行一次 TLB shootdown，shootdown 完成后再释放这批所有权。顺序不可 颠倒：先清 PTE、保留引用、完成远端 TLB 失效，最后释放物理页。RISC-V 页表库同时接入 系统级 TLB flush handler，删除调用点上重复的手工全核 flush。

3.6.3 仅向使用目标页表的处理器刷新 TLB（active-root）

每个 CPU 通过原子数组 `ACTIVE_PAGE_TABLE_ROOTS` 发布当前用户页表根。页表修改携带 `root_paddr`，RISC-V 只向仍运行该地址空间的 hart 发 SBI RFENCE，LoongArch 只向 选中 CPU 发 IPI 并执行按 VA 的 `invtlb`；root 为 0 或无法确定时保守刷新所有 CPU。 CPU 切入新 root 时先发布、再安装页表、随后本地全刷，避免并发 shootdown 漏掉目标 CPU。主要代码：`kernel/src/mm/aspace/backend/cow.rs`（`next_present_mapping`、 `UnmapGather`）、`kernel/src/mm/tlb.rs`（`ACTIVE_PAGE_TABLE_ROOTS`）。

3.7 其他优化

核心优化完成后，缺页提交路径上的引用计数、PTE 安装与 VFS 名字解析/元数据读取成为剩余热点。以下优化属于细碎改进：frame 引用计数直查、PTE/COW/Folio 快速提交，以及 VFS 目录项与 inode 元数据并发缓存。说明：VFS 相关改动在真实分支中随 7/24 核心缺页 优化包一并合入，此处按实验分支的测量顺序列出。

3.7.1 缺页提交微路径

- frame 引用计数从 BTree 查找改为按物理页号直接索引的紧凑数组（64 路分片），并把 临界区从“关中断”缩短为“仅禁止抢占”；
- missing-fault commit 直接用 PTE CAS 返回的旧值判断成功/等价/stale，删除 commit 前重复的完整页表查询；
- COW 文件页所有权使用内联槽，超出容量才进入 BTree overflow，纯匿名映射不创建文件 页所有权表；
- folio fault-around：从故障 folio 派生兄弟页，一次 canonical validation 后最多批量 CAS 安装 16 张 PTE，故障页优先、兄弟页冲突可跳过。

主要代码：`kernel/src/mm/aspace/backend/cow.rs`、 `kernel/src/mm/aspace/backend/mod.rs`。

3.7.2 VFS 目录项与 inode 元数据并发缓存

`vendor/axfs-ng-vfs/src/node/dir.rs`：目录缓存同时保存 positive/negative dentry， 已确认不存在的名字直接返回 `NotFound`；每个目录设 16 个按文件名哈希的睡眠锁，同名 并发 miss 只允许一个线程实际查询，不同名字仍可并行；文件系统 lookup 期间不持有目录 cache 写锁，通过 `cache_generation` 检查 namespace 是否改变；create/unlink/link/ rename 由目录 `mutation_lock` 串行提交，跨目录 rename 按地址顺序取锁避免死锁。`vendor/axfs-ng-vfs/src/node/mod.rs`：inode metadata cache 改为 64 路分片，同一 inode 的加载与回写使用睡眠锁合并，不同 inode 的 `stat`/属性读取不再争同一把全局缓存锁。

第 4 章 实验分析

4.1 实验环境与方法

所有正式性能数据使用统一的实验协议（`scripts/run_rv_stage_metrics.sh`）：

项目	配置
架构 / 机器	RISC-V 64，QEMU virt
vCPU / guest 内存	8 vCPU / 8 GiB
磁盘	仓库 sdcard-rv.img（raw），QEMU -snapshot
QEMU 版本	10.2.1
内核构建	make kernel-rv（正式性能）；make PROFILE=y kernel-rv（热点基线）
宿主机采样	每秒 pidstat / mpstat / iostat
正确性判定	cagent 与 minibuild 通过；BUILDSTORM_COMPILE ok=true；输出 1681000 字节；QEMU 退出码 0

表 4-1　实验环境

每个代码阶段先提交、再构建、再运行一次完整 BuildStorm，保存 `environment.txt`、`console.log`、`qemu-time-v.txt`、`host-summary.json`、`checksums.sha256` 与 `result.md`。需要说明：目前每个阶段只有一次正式样本，宿主机为桌面环境、未做 CPU 隔离，因此本文把阶段差值表述为“单次观测”，并明确哪些结论具备强归因、哪些仍需要 重复实验确认。

4.2 端到端结果

六项核心优化按累积顺序的测量结果（均关闭 guest profiler）如下：

阶段	测量提交	累积优化	**BuildStorm**	相邻变化	相对 **baseline**
00-U baseline	f360f2f0	baseline + pipe/FD 正确性修复	1524.50 s	—	—
01	31484b49	+ PCP 物理页缓存	1058.68 s	-30.56\%	-30.56\%
02	d01a0dce	+ AddrSpace 三段式	1197.38 s	+13.10\%	-21.46\%
03	0d6648b4	+ folio 页缓存	879.17 s	-26.58\%	-42.33\%
04	e1649f0a	+ 只读文件直接映射	772.47 s	-12.14\%	-49.33\%
05	d9f6d670	+ ext4 锁外 I/O 与异步 VirtIO	765.18 s	-0.94\%	-49.81\%
06	9fcb29ab	+ per-VMA mprotect	628.53 s	-17.86\%	-58.77\%

表 4-2　六项核心优化累积实验结果（unprofiled baseline 至最终版）

在相同口径下，最终版本相对 unprofiled baseline 的 BuildStorm 由 1524.50 秒降至 628.53 秒，减少 895.97 秒（-58.77%），约 2.43 倍加速。

六项核心之后的后续优化在同一实验链路中继续测量：

阶段	测量状态	累积优化	**BuildStorm**	相邻变化
07	3ddf0775-dirty	+ VFS lookup 与 inode metadata cache	648.61 s	+3.19\%
08	07b5ac82-dirty	+ 驻留页异步回收与 UnmapGather	542.27 s	-16.39\%
09	285fc139-dirty	+ active-root TLB shootdown	466.22 s	-14.02\%
10	55a2df52-dirty	+ frame/PTE/COW 缺页提交微路径	460.76 s	-1.17\%

表 4-3　后续优化实验结果

Stage 10 的 460.76 秒相对 unprofiled baseline 累计下降 69.78%。Stage 07～10 为 dirty worktree 测量（差异保存在 `source-worktree.patch`），且宿主可用内存等外部条件 存在波动，因此这四轮更适合作为“方向性证据”而非严格 clean-commit A/B。

4.3 逐项优化分析

4.3.1 PCP：单次强正向

同口径 baseline（同样携带 pipe/FD 正确性修复）补齐后，PCP 相邻单次结果为 −30.56%（1524.50 → 1058.68 秒），平均有效核数从 3.252 升至 3.536，6～8 核时间占比 从 6.23% 升至 22.38%。baseline 与 PCP 都包含相同的 pipe/FD 修复，因此退出修复 不再是变量。该结果只有一轮，30.56% 仍需重复运行确认，但方向上 PCP 是目前最强的 单项端到端证据之一。

4.3.2 AddrSpace 三段式：架构必要，本轮未获证

相邻单次结果从 1058.68 秒变为 1197.38 秒（+13.10%），并行度同步下降。该轮同时 引入了 COW commit 的远程 TLB flush 等新同步成本。当前判断是：三段式提供了后续 folio 与锁外 I/O 所需的并发框架，但这一轮的数据不能证明其独立收益，也不能只凭一次 较慢结果认定必然回退，需要重复运行区分“实现回退”与“单轮波动”。

4.3.3 folio：相邻最大降幅

相邻单次 −26.58%（1197.38 → 879.17 秒），平均有效核数 +8.61%，6～8 核时间占比 +14.91 个百分点；代价是平均 RSS +22.08%、估算读写量各增加约 43%。相比 PCP 轮的 1058.68 秒仍快 16.96%，说明大幅下降不全是上一轮较慢样本造成的。folio 用更多缓存 与批量 I/O 换取更少的逐页缺页、等待与唤醒，QEMU system CPU time 下降 50.75%、 主动上下文切换下降 78.34%。

4.3.4 只读文件直接映射：端到端、内存与 CPU 证据一致

相邻单次 −12.14%（879.17 → 772.47 秒）；QEMU user CPU time −13.70%， user + system 总 CPU time −9.13%；平均 RSS −18.56%，宿主机 minor faults −84.71%；估算读取量 −18.04%。代价是同步 TLB shootdown 使 system CPU time 上升 约 4 倍、主动上下文切换大幅增加（Stage 04 为 36,329,202 次），这为后续 unmap gather 与 active-root TLB 优化提供了直接动机。

4.3.5 ext4 锁外 I/O 与异步 VirtIO：局部 I/O 明显、端到端约 1%

相邻单次 −0.94%（772.47 → 765.18 秒），但估算写入量 −47.21%、读取量 −22.99%，整段 QEMU 文件系统输出 −47.50%。当前 BuildStorm 的磁盘等待不是主瓶颈（宿主机 iowait 约 0.02%），因此 I/O 下降没有按比例转化为墙钟收益。该轮是“ext4 锁外 I/O + 异步 VirtIO 多在途请求”两层机制的整包结果，不能拆分为各自收益；但机制本身明确， 对更慢磁盘或更高 I/O 并发负载更有价值。

4.3.6 per-VMA mprotect：证据最强的单项优化

相邻单次 −17.86%（765.18 → 628.53 秒），平均有效核数 +18.39%，中位有效核数 +24.09%，6～8 核时间占比 +11.46 个百分点，而总 CPU 时间只下降 2.58%——墙钟改善 主要来自“把原来排队逐个干的工作放回多个核并行”，与删除全局扫描和全局锁的目标一致。 历史相邻版本 A/B（`mprotect-fix-20260724`）更清楚：旧全局 `MMAP_RECORDS` 路径 1243.89 秒，per-VMA 实现 567.41 秒，下降 54.38%，`mprotect` 累计 task-time 下降约 99.3%，且调用次数没有减少。

4.3.7 后续优化的证据

- Stage 08（异步回收 + UnmapGather）：相对 Stage 07 单次 −16.39%，QEMU system CPU time −87.66%，主动上下文切换 −92.61%，总 CPU time −26.37%；回收 RFENCE 大幅 减少；
- Stage 09（active-root TLB）：相对 Stage 08 单次 −14.02%，总 CPU time −18.56%， system CPU time −70.68%；
- Stage 10（frame/PTE/COW 微路径）：相对 Stage 09 单次 −1.17%，在宿主内存更紧张、 读取量增加约 19% 的不利条件下仍略快；
- 更早时间线的独立证据：folio fault-around 相对正确性基线 582.76 → 539.38 秒 （-7.44%）；unmap gather 单轮 543.10 → 531.20 秒（-2.19%）；抢占释放快路径 531.20 → 508.49 秒（-4.27%）；当前分支最低单次记录 504.26 秒。

4.4 CPU 并行度变化

把 QEMU 进程每秒 `%CPU / 100` 记为“有效核数”。各阶段正式窗口统计如下：

阶段	平均有效核数	中位数	**P95**	**6～8** 核占比
baseline（ 无 profiler）	3.252	3.480	6.120	6.23\%
PCP	3.536	3.080	7.341	22.38\%
AddrSpace 三段式	3.313	2.985	6.932	17.70\%
folio	3.598	2.050	7.770	32.61\%
只读直接映射	3.738	3.120	7.584	32.21\%
ext4 锁外 I/O	3.725	3.030	7.605	31.46\%
per-VMA mprotect	4.410	3.760	7.940	42.93\%

表 4-4　各阶段 CPU 并行度（正式 BuildStorm 窗口）

最终版相对无 profiler baseline：平均有效核数 +35.62%，P95 从 6.120 核升到 7.940 核，6～8 核时间增加 36.70 个百分点。峰值的提升有限，真正改变的是“能维持高并行 多长时间”。这组数据支持本文的核心理解：优化不是把单核算得更快，而是减少全局锁、逐页等待、重复复制与全表扫描，让更多阶段能更长时间同时使用多个核。

4.5 内存与 I/O

阶段	平均 **RSS**（**KiB**）	最大 **RSS**（**KiB**）	估算读取（**KiB**）	估算写入（**KiB**）
baseline（无 profiler）	2551119	2921592	469282	390434
PCP	2594330	3070044	523044	362744
AddrSpace 三段式	2597466	3089976	448212	373526
folio	3171058	3927916	639157	535631
只读直接映射	2582383	3141188	523871	495356
ext4 锁外 I/O	2595280	3084728	403419	261508
per-VMA mprotect	2564150	3124756	400715	246463

表 4-5　各阶段内存与宿主机 I/O 估算（正式窗口）

最终版相对 baseline：平均 RSS 基本持平（+0.51%），最大采样 RSS +6.95%；估算读取 量 −14.61%，估算写入量 −36.87%。中间的 folio 阶段主动用更大缓存和批量 I/O 换取 性能（RSS 与 I/O 一度上升），只读直接映射省掉私有页分配与复制后 RSS 明显回落， ext4 锁外路径又显著减少重复写入。这些数字来自宿主机对 QEMU 的一秒采样近似积分， 且使用 `-snapshot`，不能等价为 guest ext4 的精确逻辑 I/O。

4.6 历史第一轮整包数据

2026-07-24 第一轮整包优化（`ea78c094`）的历史测量：正式 BuildStorm 从约 2485 秒 降至 1178.57 秒（-52.57%）。同轮 guest 定向 task-time 变化为：

指标	优化前	优化后	降幅
page-cache inclusive	9343.23 s	37.52 s	—
ext4 inclusive	682.54 s	42.18 s	-93.82\%
同步块读取	175.67 s	1.85 s	-98.95\%
精确 ext4 全局锁	218.92 s	0.69 s	-99.68\%

表 4-6　2026-07-24 整包优化关键路径 task-time 变化

该轮同时包含多项缺页、页缓存、ext4 与分配器修改，因此 52.57% 是整条链路的组合 收益，不能倒算给某一项优化。这也是实验数据仓库建立后，重新按阶段逐项测量（表 4-2） 的原因。

4.7 结果可信度与限制

- 单样本限制：表 4-2 的每个阶段目前只有一次正式运行，-58.77% 是当前同口径单次 结果；稳定的加速比应以至少三轮运行的中位数为准。相邻阶段差值更不能直接当作每项 优化的稳定独立贡献；
- 环境噪声：桌面宿主机未做 CPU 隔离，Stage 07 与 Stage 06 之间宿主可用内存相差 约 4.84 GiB，会放大单轮波动；
- 归因边界：Stage 05 是 ext4 与异步 VirtIO 的整包；Stage 07～10 为 dirty worktree 测量；folio 的核心收益与 fault-around 的 7.44% 已按时间线分开记录；
- 正确性与性能分离：远程 SBI RFENCE、TLB shootdown 等属于 SMP 正确性要求，不能为 追求旧数字而删除；性能评价始终以“同口径、同正确性基线”为前提。

第 5 章 AI 使用说明与可复现步骤

5.1 AI 使用说明

本项目的开发过程使用了大型语言模型（AI 编程助手）作为辅助工具，主要用于以下场景：

- 代码阅读与热点定位：让 AI 阅读内核源码与 profiler 汇总（火焰图、路径探针）， 协助梳理“缺页 → 地址空间锁 → 页缓存 → ext4 → 块设备”调用链，定位锁竞争与热点 函数；
- 方案设计与代码生成：在人工确定优化方向和约束后，由 AI 生成关键优化代码（如 PCP 页缓存、三段式缺页框架、folio 状态机、per-VMA `mprotect`、extent cache 等）， 团队成员逐行 review 后合入；
- 正确性审查：AI 辅助检查 Rust 所有权/生命周期、锁顺序、引用计数与 TLB 失效顺序 等容易出错的地方（例如 folio `invalidate_lock` 等待顺序、`FolioPageRef` 生命周期、 UnmapGather 的“先清 PTE → 远端 TLB 失效 → 再释放页”顺序）；
- 实验与数据处理：AI 辅助编写/检查实验脚本，解析 `console.log` 与 `host-summary.json`，生成阶段对比表与报告；
- 文档撰写：本报告由 AI 辅助整理实验数据与设计说明，所有数字均来自仓库中实际 保存的实验记录，未采用 AI 推测的性能数值。

使用原则：所有 AI 生成的代码必须通过人工审查、单元测试与完整 BuildStorm 回归后才 合入；涉及性能的结论只使用实际测量数据；失败的尝试（例如 COW HashMap、`memset` 字宽改写等）也保留记录，避免重复踩坑。

5.2 可复现步骤

5.2.1 1. 环境准备

- 主机：x86_64 Linux（本实验使用桌面宿主机，QEMU 10.2.1），至少 8 个物理核；
- 工具链：Rust nightly-2025-05-20；`qemu-system-riscv64`；
- 磁盘镜像：仓库 `sdcard-rv.img`（评测镜像），运行参数与 `scripts/run_rv.sh` 一致，并额外使用 `-snapshot` 保证每次从相同状态启动；
- 仓库：MossOS 内核仓库，实验链路位于 `experiment-data-organize` 分支。

5.2.2 2. 构建与运行

```
# 切换实验分支
git switch experiment-data-organize

# 构建内核（正式性能测量；热点定位使用 make PROFILE=y kernel-rv）
make kernel-rv

# 运行完整 BuildStorm 并收集原始数据
./scripts/run_rv_stage_metrics.sh docs/experiment_data/<stage>
```

每个阶段目录应包含：`environment.txt`（提交/内核/镜像/宿主机身份）、 `console.log`（完整 guest 输出）、`qemu-time-v.txt`、`host-qemu-pidstat.log`、`host-summary.json`、`checksums.sha256` 与 `result.md`（结果与对比）。

5.2.3 3. 分阶段复现

从 baseline 按依赖顺序逐个应用提交并测量：

阶段	测量基点	内容	实验数据目录
00	94d3b47a	profiling baseline（热点定位，PROFILE=y）	docs/experiment_data/00-baseline/
01	ee400aa8	per-CPU 物理页缓存	docs/experiment_data/01-pcp/
02	d01a0dce	AddrSpace 三段式缺页	docs/experiment_data/02-aspace-three-phase/
03	2fd0f7c1	folio 文件页缓存	docs/experiment_data/03-folio/
04	e1649f0a	只读文件页直接映射	docs/experiment_data/04-readonly-file-direct-map/
05	d9f6d670	ext4 锁外 I/O 与异步 VirtIO 队列	docs/experiment_data/05-ext4-lock-outside-io/
06	9fcb29ab	per-VMA mprotect	docs/experiment_data/06-mprotect-per-vma/
07	3ddf0775	VFS lookup 与 inode metadata cache	docs/experiment_data/07-vfs-lookup-and-metadata-cache/
08	07b5ac82	驻留页异步回收与 UnmapGather	docs/experiment_data/08-aspace-resident-reclaim-unmap-gather/
09	285fc139	active-root TLB shootdown	docs/experiment_data/09-riscv-tlb-active-root/
10	55a2df52	frame/PTE/COW 缺页提交微路径	docs/experiment_data/10-frame-pte-cow-commit/

表 5-1　分阶段复现代码提交

说明：表 4-2 中 00-U baseline 的测量提交为 `f360f2f0`（`94d3b47a` + pipe/FD 正确性 修复 + 测量脚本）；Stage 01 的测量提交 `31484b49` 为 PCP 实现 `ee400aa8` 叠加同一 修复。Stage 07～10 的测量基点为 dirty worktree，实际代码差异保存在对应目录的 `source-worktree.patch` 中，并已分别提交（VFS 为 `07b5ac82`、回收/UnmapGather 为 `285fc139`、active-root TLB 为 `55a2df52`，frame/PTE/COW

微路径随后合入）。未配置 profiler 的 baseline 数值与逐阶段对比见 `docs/experiment_data/overall-comparison.md`；每阶段的详细结果见对应目录的 `result.md`。

5.2.4 4. 正确性判定

每个阶段在统计性能前先确认：cagent 测试组与 minibuild 通过；`BUILDSTORM_COMPILE` 报告 `ok=true`；输出架构为 `riscv64`、产物大小为 1,681,000 字节；Cargo/axbuild 与 测试组 `END` 标记齐全；QEMU 正常自动退出且退出码为 0；内核与镜像 SHA-256 已校验。

5.2.5 5. 建议的测量纪律

- 每个阶段至少运行 3 次，报告中位数并保留每次原始值；
- 下一项优化从已验证的上一个阶段开始，不混跑；
- 正式性能与热点 profiling 分开：`make kernel-rv` 的结果不混入 profiler 开关差异；
- 单项归因使用紧邻提交的 A/B，避免把整包收益倒算给某一项。

第 6 章 总结与展望

本文档完整记录了 MossOS 针对 BuildStorm 测例的问题定位、优化设计与实验验证过程。核心结论如下：

- 瓶颈定位明确：与 Linux 的同口径对比证明差距集中在文件缺页、页缓存、同步 I/O 与 ext4 锁等待，而非编译计算本身；
- 优化方向正确：六项核心优化按依赖顺序实施，共同缩短同一条“文件缺页 → 物理页 分配 → 页缓存 → ext4 → 块设备 → mprotect”链路；
- 端到端效果显著：同口径单次测量中，六项核心优化使 BuildStorm 从 1524.50 秒降至 628.53 秒（-58.77%，约 2.43 倍加速）；继续完成后续优化后降至 460.76 秒；
- 证据分级透明：per-VMA `mprotect`、folio、PCP、只读直接映射具备强单次正向证据；AddrSpace 三段式与 ext4 锁外 I/O 被记录为“架构/机制必要但单轮端到端收益未充分 证明”，避免把整包收益错误归因。

下一步工作包括：为每个阶段补齐至少三轮运行的中位数；为尚未隔离的项目补做紧邻 clean-commit A/B；增加 guest 内低开销计数器（PCP hit/miss、folio loader/waiter、extent cache hit、mprotect 扫描 VMA 数、RFENCE 次数等）以精确拆分收益；在 LoongArch 平台重复相同协议，验证跨架构一致性。